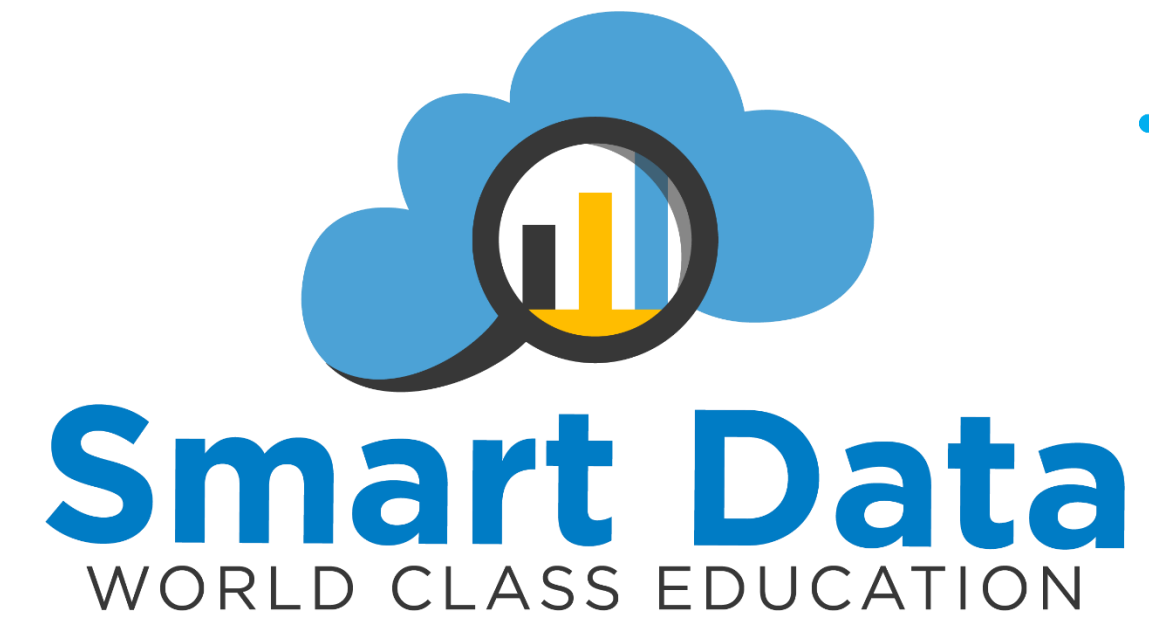
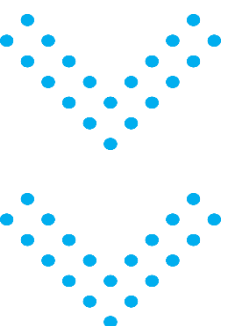
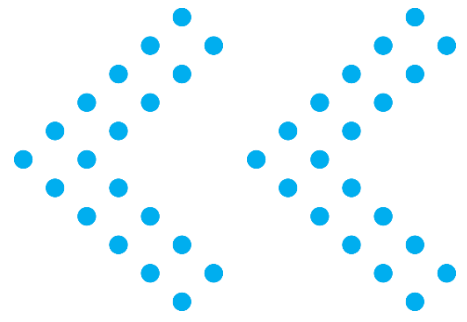
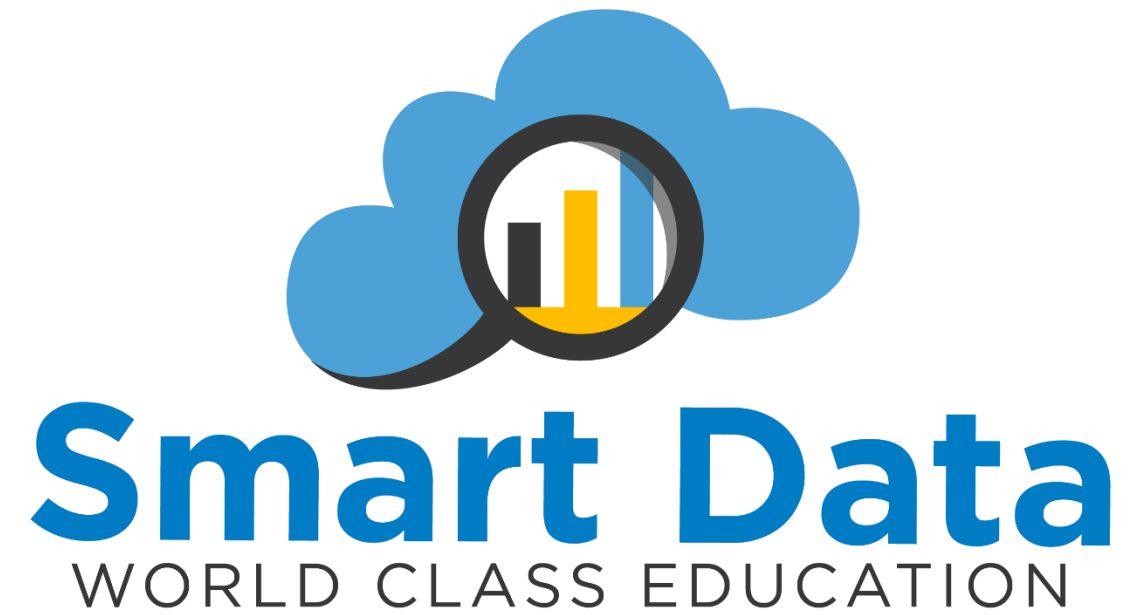




# MLOps: Del Modelo al entorno Productivo





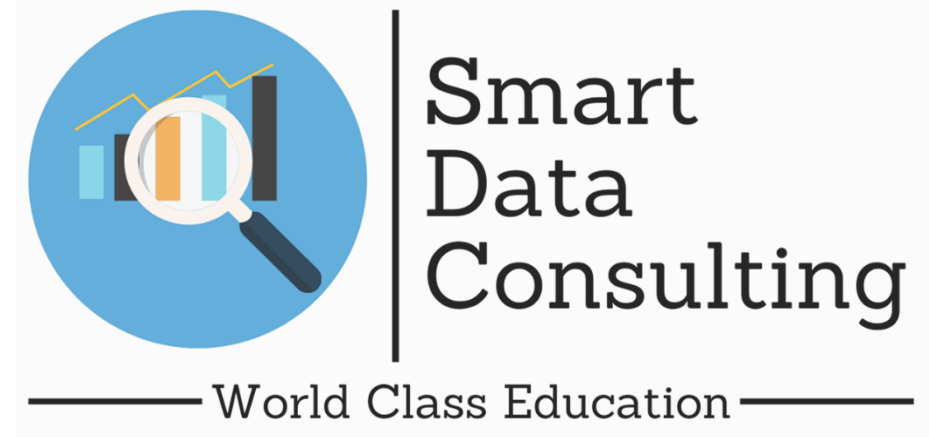


# Pipelines reproducibles

**Instructor:**

**Julio César Bernal Fernández**

# Contenido de la Presentación



**01**    ¿Qué es un Pipeline en ML?

**02**    Separación por Etapas

**03**    Ingestión de Datos

**04**    Ingeniería de Features

**05**    Entrenamiento del Modelo

**06**    Evaluación del Modelo

**07**    Validación de Datos

**08**    Ejecución Secuencial con Scripts

# ¿Qué es un Pipeline en ML?

 [scikit-learn.org/stable/modules/compose.html](https://scikit-learn.org/stable/modules/compose.html)

Un pipeline de Machine Learning es una secuencia automatizada y estructurada de pasos que transforma datos crudos en predicciones o modelos entrenados. Cada paso es independiente, trazable y reproducible.

## Automatización

Elimina pasos manuales  
propensos a errores

## Reproducibilidad

Mismo input → mismo  
output, siempre

## Modularidad

Cada etapa es testeable  
de forma independiente

## Escalabilidad

Fácil extender con  
nuevas etapas

# Pipeline vs Flujo Ad-hoc

## ✗ Flujo Ad-hoc

- Scripts sueltos sin orden definido
- Difícil reproducir resultados
- Estado global compartido
- Depuración compleja y costosa
- Cambios rompen todo el sistema



## ✓ Pipeline Estructurado

- ✓ Etapas claramente definidas
- ✓ 100% reproducible
- ✓ Sin efectos secundarios
- ✓ Testeable por etapa
- ✓ Mantenible y extensible

# ¿Por qué importa la reproducibilidad?

Un modelo que no se puede reproducir no es confiable. La reproducibilidad es la base del ML en producción.

**87%**

de experimentos ML  
no son reproducibles

**3x**

más rápido debugging  
con pipelines

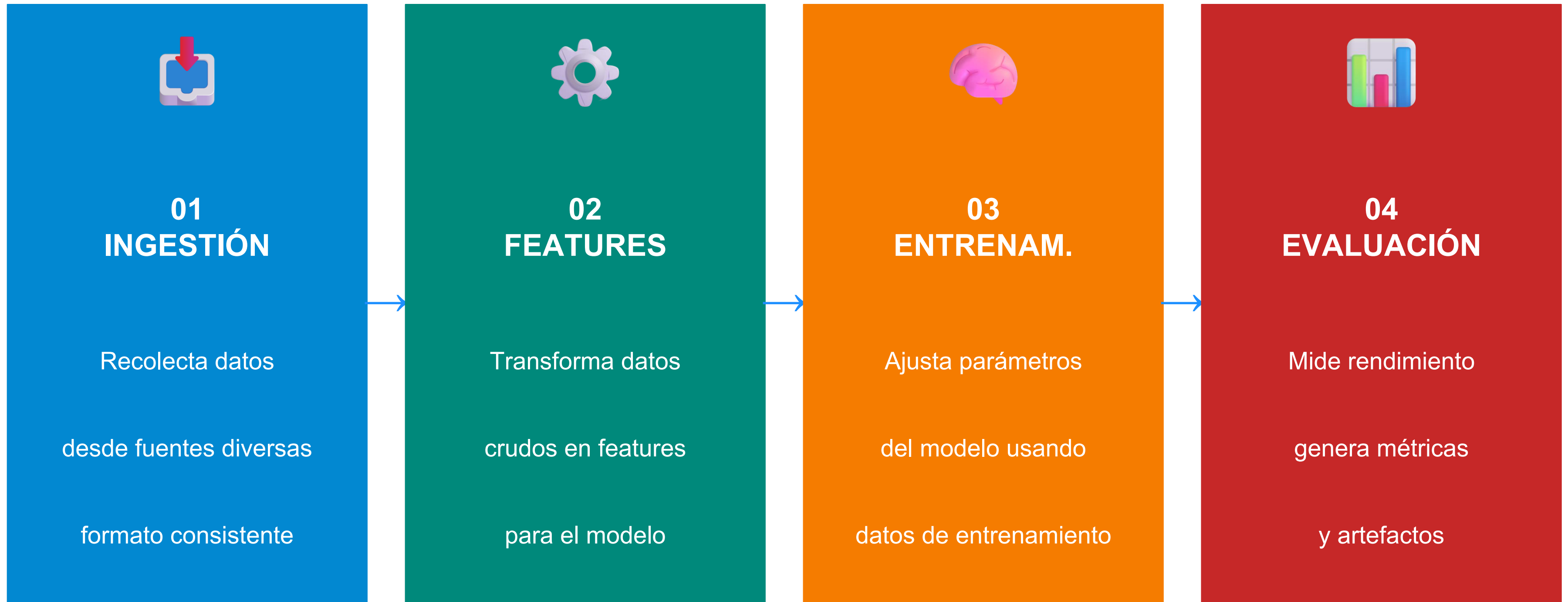
**60%**

de costo evitado  
en re-entrenamiento

La reproducibilidad garantiza que los experimentos sean validables por el equipo, que los modelos puedan ser auditados y que el mantenimiento del sistema sea sostenible en el tiempo.

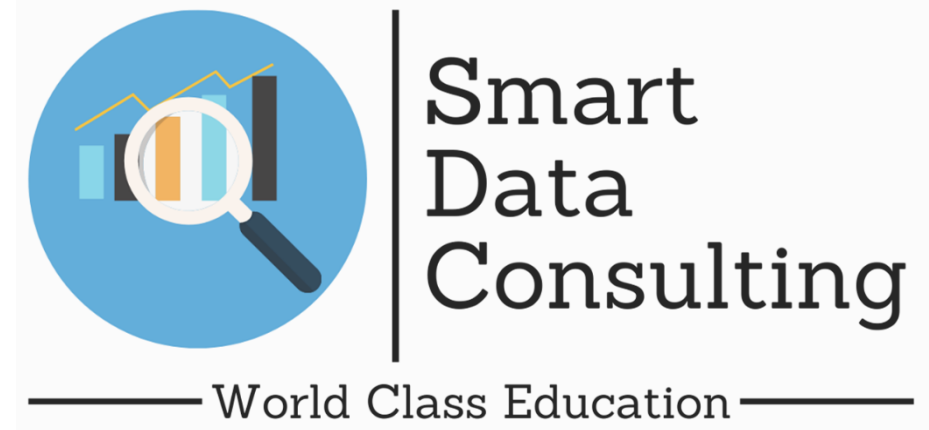


# Las 4 Etapas del Pipeline



*Cada etapa recibe entradas claras y produce salidas verificables*

# Etapa 1 · Ingestión de Datos



Recolecta y centraliza los datos desde diversas fuentes en un formato consistente y versionado.

## Fuentes de datos

Bases de datos, APIs REST, archivos CSV/JSON, streams en tiempo real

## Versionado

Cada dataset recibe un hash MD5/SHA256 único para trazabilidad completa

## Logging

Registro de cuándo, desde dónde y cuántas filas se ingirieron

## Pre-limpieza

Eliminación de duplicados obvios, corrección de encoding, tipos básicos



# Ingestión · Implementación en Python

```
# ingestion/ingest.py
import pandas as pd, hashlib, json, datetime

def load_raw_data(source_path: str) -> pd.DataFrame:
    """Carga datos y registra metadata de ingestión."""
    df = pd.read_csv(source_path)
    metadata = {
        "source": source_path,
        "rows": len(df),
        "hash": hashlib.md5(open(source_path, "rb").read()).hexdigest(),
        "timestamp": datetime.datetime.utcnow().isoformat()
    }
    with open("data/raw/metadata.json", "w") as f:
        json.dump(metadata, f)
    return df
```

💡 **Clave:** guardar hash del archivo garantiza que siempre se entrene con exactamente los mismos datos

# Etapa 2 · Ingeniería de Features

Transforma los datos crudos en representaciones numéricas útiles para el modelo.

## Transformaciones

- Normalización / escala
- Encoding categórico
- Imputación de nulos
- Binning / discretización
- Selección de variables

## Features Derivadas

- Ratios entre columnas
- Fechas → día/mes/año
- Lag features (series)
- Interacciones polinomiales
- Embeddings textuales

## Buenas Prácticas

- Fit solo en train set
- Guardar transformers
- Documentar cada feature
- Pipeline con sklearn
- Versionar el transformer

# Features · Pipeline con scikit-learn

```
# features/build_features.py
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OrdinalEncoder
from sklearn.compose import ColumnTransformer
import joblib

def build_feature_pipeline(num_cols, cat_cols):
    numeric = Pipeline([('scaler', StandardScaler())])
    categorical = Pipeline([('enc', OrdinalEncoder())])
    transformer = ColumnTransformer([
        ('num', numeric, num_cols),
        ('cat', categorical, cat_cols),
    ])
    return transformer

# Fit SOLO en train – nunca en test (evita data leakage)
transformer = build_feature_pipeline(num_cols, cat_cols)
transformer.fit(X_train)
X_train_t = transformer.transform(X_train)
X_test_t = transformer.transform(X_test) # usa parámetros de train
joblib.dump(transformer, 'models/transformer.pkl')
```

# Etapa 3 · Entrenamiento del Modelo

Ajusta los parámetros del modelo usando los features de entrenamiento de forma reproducible.

## Semilla aleatoria fija

`random_state=42` en todos los estimadores y operaciones aleatorias

## Split reproducible

`train_test_split` con `stratify` y semilla fija garantizan mismo split

## Versionado de modelos

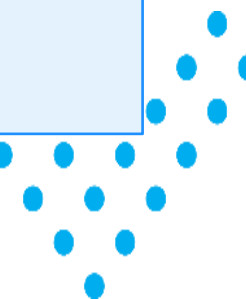
Guardar modelo con timestamp + hash de los datos de entrenamiento

## Registro de hiperparámetros

Logear TODOS los parámetros del experimento (MLflow/JSON)

## Cross-validation

K-fold estratificado para estimación robusta del rendimiento real





# Entrenamiento · Implementación reproducible

```
# training/train.py
import joblib, json, datetime
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

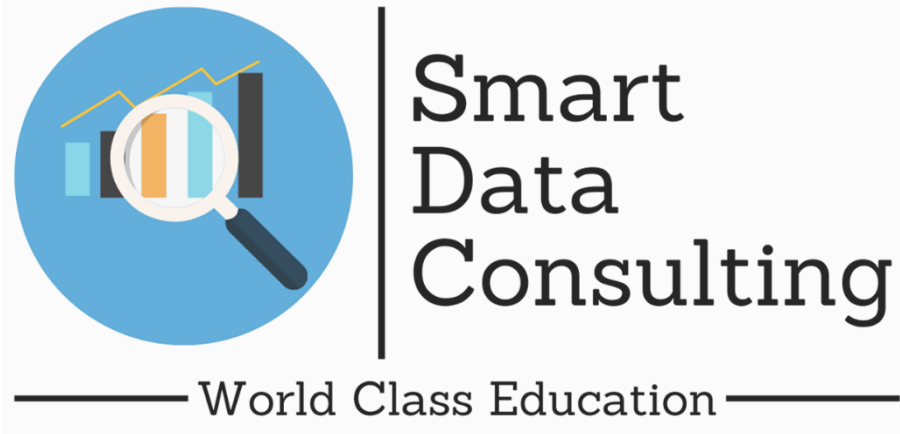
SEED = 42  # Fijar SIEMPRE la semilla para reproducibilidad

def train_model(X, y, params: dict):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=SEED, stratify=y
    )
    model = RandomForestClassifier(random_state=SEED, **params)
    model.fit(X_train, y_train)

    ts = datetime.datetime.utcnow().strftime('%Y%m%d_%H%M%S')
    joblib.dump(model, f'models/model_{ts}.pkl')
    json.dump(params, open(f'models/params_{ts}.json', 'w'), indent=2)
    return model, X_test, y_test
```

💡 El timestamp + versión de datos en el nombre del archivo permite rastrear exactamente qué modelo se generó con qué datos

# Etapa 4 · Evaluación del Modelo



Mide el rendimiento del modelo y genera artefactos de evaluación persistentes y comparables.

|          |                                                |                  |                                          |
|----------|------------------------------------------------|------------------|------------------------------------------|
| Accuracy | Proporción de predicciones correctas totales   | Precision        | Exactitud en las predicciones positivas  |
| Recall   | Cobertura de los casos positivos reales        | F1-Score         | Media armónica de Precision y Recall     |
| ROC-AUC  | Área bajo la curva ROC — discriminación global | Confusion Matrix | Tabla completa: TP, TN, FP, FN por clase |

# Evaluación · Implementación con artefactos

```
# evaluation/evaluate.py
from sklearn.metrics import classification_report, confusion_matrix
import json, matplotlib.pyplot as plt, seaborn as sns

def evaluate_model(model, X_test, y_test, output_dir='reports/'):
    y_pred = model.predict(X_test)
    report = classification_report(y_test, y_pred, output_dict=True)

    # Guardar métricas como JSON (versionable con Git/DVC)
    json.dump(report, open(f'{output_dir}metrics.json', 'w'), indent=2)

    # Guardar confusion matrix como imagen persistente
    cm = confusion_matrix(y_test, y_pred)
    fig, ax = plt.subplots(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax)
    ax.set_xlabel('Predicho'); ax.set_ylabel('Real')
    plt.tight_layout()
    plt.savefig(f'{output_dir}confusion_matrix.png', dpi=150)
    plt.close()
    return report
```

# Validación de Datos · Errores Silenciosos

[greatexpectations.io/blog/ml-pipeline-data-quality](https://greatexpectations.io/blog/ml-pipeline-data-quality) | [whylogs.readthedocs.io](https://whylogs.readthedocs.io)

Los errores silenciosos no lanzan excepciones pero producen resultados incorrectos. El modelo se entrena sin problemas aparentes, pero con datos corruptos.

## 🔴 Data Leakage

Información del futuro contamina el entrenamiento → métricas infladas

## 🟠 Distribución cambiada

Los datos de producción difieren del entrenamiento → degradación silenciosa

## 🟡 Nulos inesperados

Columnas que no deberían tener nulls los tienen → imputación errónea

## 🔴 Tipos incorrectos

Columna numérica llega como string → el scaler devuelve NaN sin error

## 🟠 Cardinalidad inesperada

Categorías nuevas en producción no vistas en entrenamiento

# Validación · Tipos de Verificaciones

## Esquema

- Columnas presentes y correctas
- Tipos de datos esperados
- Sin columnas extra inesperadas

## Valores

- Rangos válidos por columna
- Cardinalidad de categóricos
- Proporción de nulos aceptable

## Distribución

- Comparar con dataset de referencia
- Detección de drift estadístico
- Skewness y kurtosis

## Integridad

- Claves foráneas válidas
- Sin duplicados en ID primario
- Consistencia temporal en series



# Validación · Código con Pandera

```
# validation/validate_data.py
import pandera as pa
from pandera import Column, DataFrameSchema, Check

schema = DataFrameSchema({
    "age": Column(int, Check.between(0, 120), nullable=False),
    "income": Column(float, Check.greater_than(0), nullable=False),
    "gender": Column(str, Check.isin(["M", "F", "O"]), nullable=False),
    "target": Column(int, Check.isin([0, 1]), nullable=False),
})

def validate(df):
    try:
        schema.validate(df, lazy=True) # lazy=True reporta TODOS los errores
        print('✅ Datos válidos')
        return True
    except pa.errors.SchemaErrors as e:
        print('❌ Errores encontrados:')
        print(e.failure_cases) # DataFrame con los casos fallidos
        raise # Detiene el pipeline – falla rápido
```

💡 **lazy=True es clave: sin él, Pandera detiene en el primer error y puedes no ver el resto de los problemas**

# Ejecución Secuencial · Estructura del Proyecto

## 📁 Estructura recomendada

```
project/
├── data/
│   ├── raw/
│   └── processed/
├── src/
│   ├── ingestion.py
│   ├── features.py
│   ├── train.py
│   └── evaluate.py
├── models/
├── reports/
├── validation/
│   └── validate_data.py
└── run_pipeline.py
```

## ✅ Beneficios de esta estructura

- Reproducción completa con un solo comando
- Fácil integración con CI/CD (GitHub Actions)
- Claro orden de dependencias entre etapas
- Testeable componente a componente
- Compatible con Make, Airflow, Prefect
- Separación clara datos / código / modelos

# Ejecución Secuencial · run\_pipeline.py

```
# run_pipeline.py – orquestador principal
import subprocess, sys, logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s %(message)s')
log = logging.getLogger('pipeline')

STEPS = [
    ('Validación', 'validation/validate_data.py'),
    ('Ingestión', 'src/ingestion.py'),
    ('Features', 'src/features.py'),
    ('Training', 'src/train.py'),
    ('Evaluation', 'src/evaluate.py'),
]

for step_name, script in STEPS:
    log.info(f'► Iniciando: {step_name}')
    result = subprocess.run(
        [sys.executable, script],
        capture_output=True, text=True
    )
    if result.returncode != 0:
        log.error(f'❌ Falló: {step_name}\n{result.stderr}')
        sys.exit(1) # Falla rápido → errores silenciosos evitados
    log.info(f'✅ Completado: {step_name}')

log.info('🎉 Pipeline completo ejecutado con éxito')
```

# Herramientas del Ecosistema ML

## MLflow

*Tracking*

Registro de experimentos, métricas e hiperparámetros.

 [mlflow.org](https://mlflow.org)

## DVC

*Versionado*

Control de versiones para datasets y modelos.

 [dvc.org](https://dvc.org)

## Pandera

*Validación*

Validación declarativa de DataFrames pandas.

 [pandera.readthedocs.io](https://pandera.readthedocs.io)

## Great Expectations

*Calidad datos*

Suite completa de validación y documentación de datos.

 [greatexpectations.io](https://greatexpectations.io)

## Apache Airflow

*Orquestación*

Scheduler de DAGs para pipelines complejos.

 [airflow.apache.org](https://airflow.apache.org)

## Makefile

*Automatización*

Orquestación simple con dependencias en shell.

 [gnu.org/software/make/](https://gnu.org/software/make/)

# Mejores Prácticas Consolidadas

 [mlops.community](https://mlops.community) | [google.com/machine-learning/crash-course](https://google.com/machine-learning/crash-course)



Smart  
Data  
Consulting

World Class Education

01

Versiona datasets y modelos como código (DVC, Git LFS)

02

Fija semillas aleatorias en todos los puntos del código

03

Valida esquema de datos antes de cada etapa del pipeline

04

Falla rápido: usa exit codes, no silencies excepciones

05

Loguea metadata: fuente, timestamp, hash del dataset

06

Separa fit/transform: NUNCA fittees en el test set

07

Guarda todos los artefactos: transformers, modelos, métricas

08

Documenta cada etapa con docstrings y README actualizado



# Referencias y Recursos

## scikit-learn — Pipelines y Preprocesamiento

[🔗 scikit-learn.org/stable/modules/compose.html](https://scikit-learn.org/stable/modules/compose.html)

## MLflow — Tracking de Experimentos

[🔗 mlflow.org/docs/latest/tracking.html](https://mlflow.org/docs/latest/tracking.html)

## Great Expectations — Calidad de Datos

[🔗 docs.greatexpectations.io/docs/](https://docs.greatexpectations.io/docs/)

## Cookiecutter Data Science — Estructura de Proyectos

[🔗 cookiecutter-data-science.drivendata.org](https://cookiecutter-data-science.drivendata.org)

## NeurIPS 2015 — Hidden Technical Debt in ML Systems

[🔗 papers.nips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf](https://papers.nips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf)

## Martin Fowler — CD4ML (Continuous Delivery for ML)

[🔗 martinfowler.com/articles/cd4ml.html](https://martinfowler.com/articles/cd4ml.html)

## Pandera — Validación de DataFrames

[🔗 pandera.readthedocs.io/en/stable/](https://pandera.readthedocs.io/en/stable/)

## DVC — Versionado de Datos y Modelos

[🔗 dvc.org/doc/start](https://dvc.org/doc/start)

## Apache Airflow — Orquestación de Pipelines

[🔗 airflow.apache.org/docs/](https://airflow.apache.org/docs/)

## MLOps Community

[🔗 mlops.community](https://mlops.community)